

Despliegue con Docker y Railway

Contenedores, Multi-Stage Builds y Cloud
Hosting para NestJS y PostgreSQL

Facultad de Ingeniería — Desarrollo de Entornos Digitales Web — Semana 9

01. Fundamentos del Despliegue Web

¿Qué Significa Desplegar en la Web?

Del Entorno Local al Mundo Real

Desplegar es trasladar nuestro software desde la máquina del desarrollador (`localhost`) hacia servidores en la nube de alta disponibilidad accesibles permanentemente las 24/7.



Disponibilidad Global Continua:

Cientes HTTP, frontends y apps móviles pueden consultar la API en cualquier momento sin depender de nuestra laptop.



Persistencia y Red en la Nube:

El backend se enlaza con bases de datos gestionadas mediante redes privadas seguras y configuraciones dinámicas.

¿Qué es Desplegar?

De un entorno local aislado a un servidor en la nube accesible 24/7 por el mundo



El Desafío de la Inconsistencia

El Síndrome "En mi máquina sí funciona"

Las discrepancias entre computadores de desarrollo y servidores de producción representan la causa principal de fallos en despliegues tradicionales.



Fricción por Entorno Manual:

Diferentes versiones de Node.js, dependencias globales no instaladas y diferencias entre Windows, macOS y Linux.



Estandarización con Contenedores:

Docker encapsula el código, dependencias, runtime y configuraciones en una unidad idéntica y reproducible.

¿Por Qué Docker? Paridad Local vs. Producción

Eliminando la discrepancia de entornos mediante empaquetado idéntico

● SIN DOCKER (CAOS)

WIN

Laptop Dev

Node: v22.1.0

Status: Funciona OK

LINUX

Servidor Cloud

Node: v18.4.0

SyntaxError / Crash



"En mi máquina sí funcionaba..."

Rutas diferentes en Windows vs Linux (\ vs /)

Librerías compiladas con versiones incompatibles

PostgreSQL local no coincide con la base remota

● CON DOCKER (ESTÁNDAR)

W

Windows

Contenedor

100% Igual

M

macOS

Contenedor

100% Igual

R

Railway

Contenedor

100% Igual

Multi-Stage Builds: Eficiencia y Seguridad

Separar Construcción de Ejecución

Un contenedor de producción no debe cargar con compiladores de TypeScript, linters ni herramientas innecesarias:



1. Etapa Builder (Horno de Compilación):

Descarga herramientas de desarrollo y compila TypeScript a JavaScript puro (`dist/`). Pesa ~1.2 GB pero es temporal.



2. Etapa Runner (Cápsula Ligera):

Parte de Alpine Linux limpio, copia solo `dist/` y dependencias de producción. Imagen ultraligera de ~140 MB con mínima superficie de ataque.

El Embudo de Optimización Multi-Stage

Cómo Docker filtra y descarta herramientas de compilación para dejar un contenedor mínimo en la nube



Orquestación Local y Variables

DESARROLLO LOCAL

Docker Compose

Un solo comando levanta simultáneamente PostgreSQL y NestJS sin ensuciar el sistema operativo anfitrión con servicios residentes.

RED INTERNA

Service Discovery

El backend se conecta a la base de datos usando el nombre del servicio (`db:5432`) a través de un puente de red virtual privado.

CONFIGURACIÓN

Variables de Entorno

Aislamiento estricto según Twelve-Factor App.
Credenciales protegidas en `.env` fuera de Git con plantilla `.env.example`.

PERSISTENCIA

Volúmenes de Datos

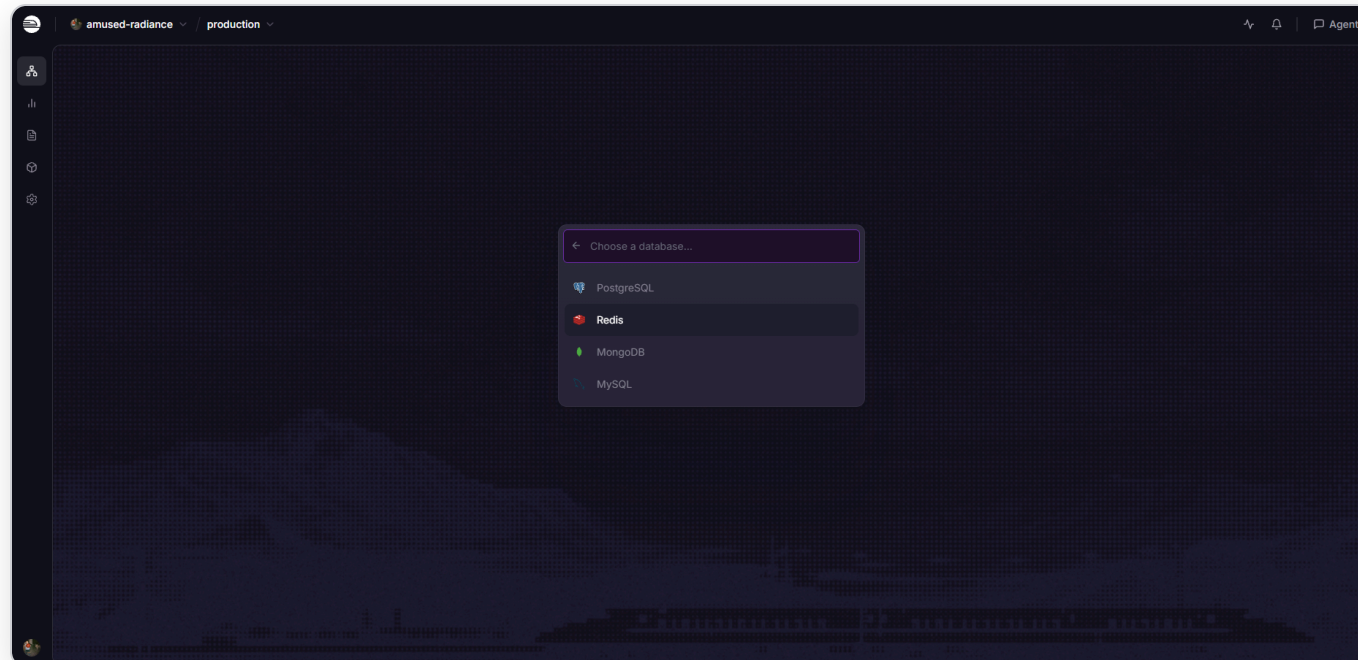
Almacenamiento persistente en disco (`pgdata`). Las tablas y registros sobreviven al reinicio o destrucción de los contenedores.

02. Despliegue en la Nube con Railway

Paso 1: Provisión de PostgreSQL en Railway

Paso 1

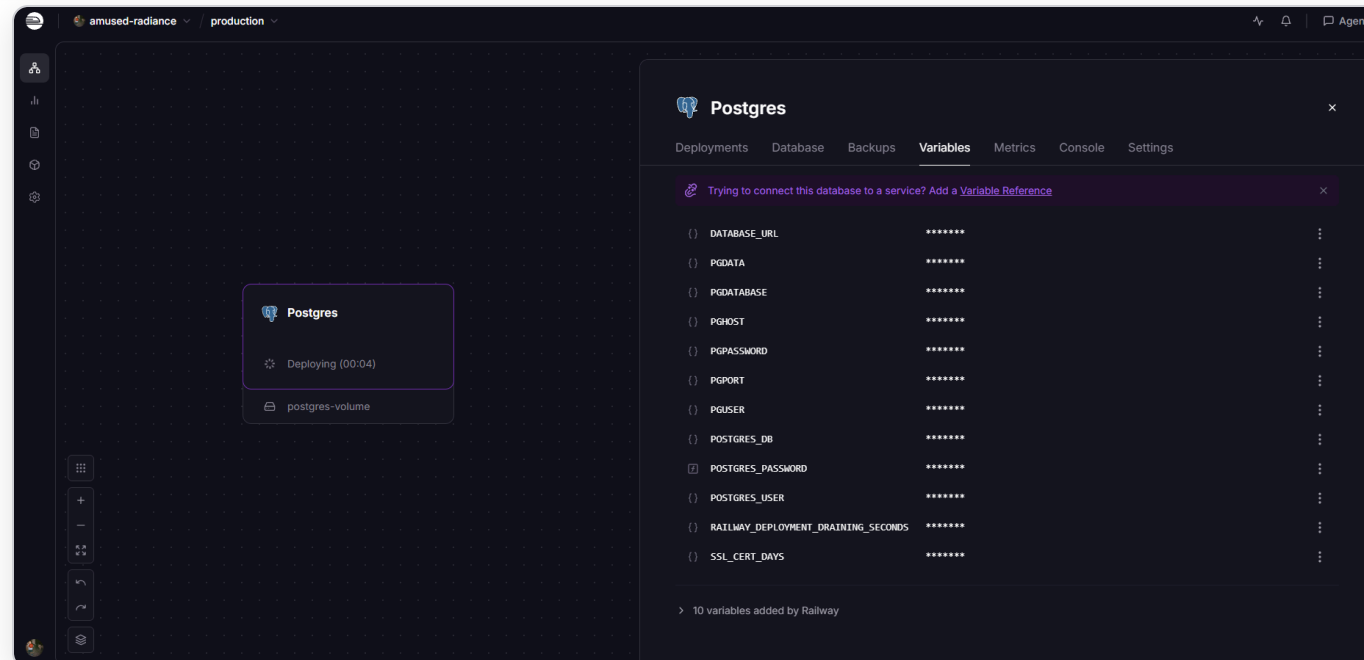
En el dashboard del proyecto pulsamos **+ New** → **Database** → **Add PostgreSQL**. Railway aprovisiona al instante una instancia administrada en la nube con red privada y almacenamiento SSD dedicado.



Paso 2: Inspección de Variables de PostgreSQL

Paso 2

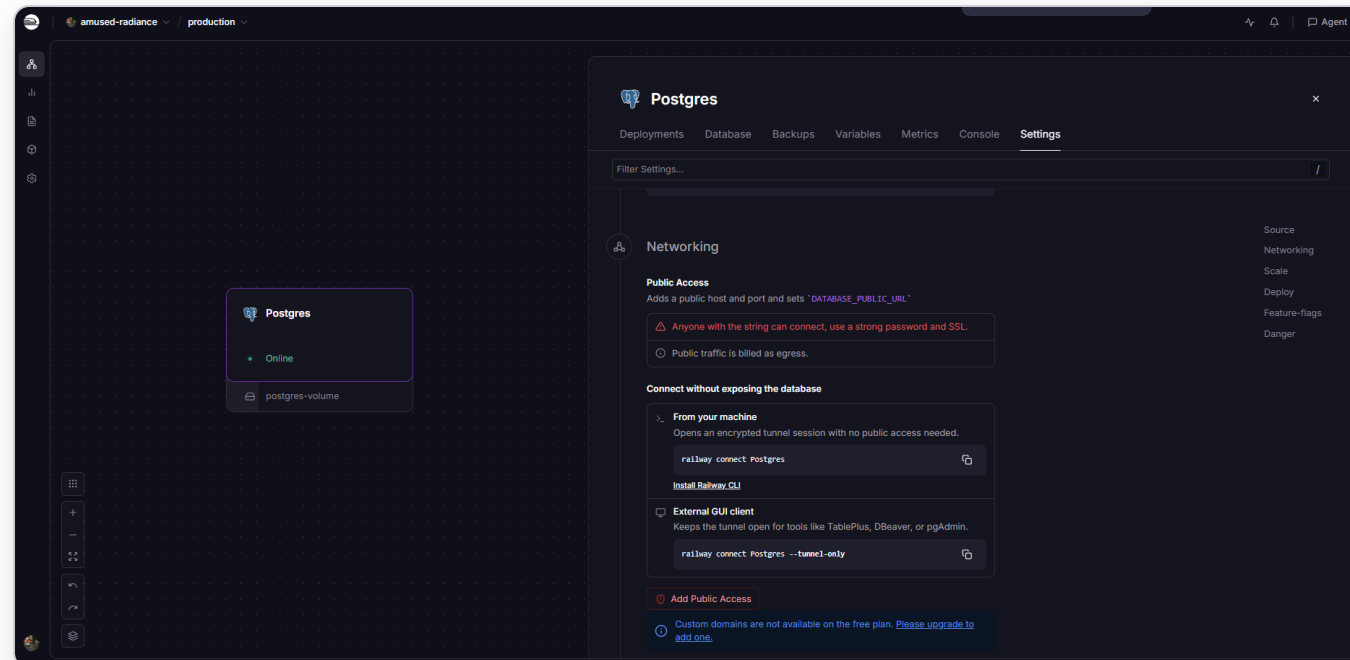
En la pestaña **Variables** de PostgreSQL se configuran automáticamente `PGHOST`, `PGPORT`, `PGUSER`, `PGPASSWORD` y `DATABASE_URL`. Estas variables se referencian internamente en el backend.



Paso 3: Acceso Público TCP para DBeaver

Paso 3

Para gestionar esquemas y tablas desde nuestro computador con DBeaver, ingresamos a **Settings** → **Networking** y activamos **Public Networking** para obtener un dominio proxy público y puerto TCP externo.



Resolución de Red: Fallback DNS ISP

¿El dominio público no conecta en DBeaver?

Algunos proveedores de internet (ISP) locales cuentan con servidores DNS lentos o bloqueos de resolución en subdominios de proxies como `*.proxy.rlwy.net`.



Diagnóstico con nslookup:

Obtenemos la dirección IP numérica pública directa asociada al proxy de Railway ejecutando una consulta DNS en la terminal.



Conexión Inmediata en DBeaver:

Reemplazamos el nombre de host por la IP obtenida (ej. `66.33.22.220`) manteniendo el puerto TCP asignado por Railway.

WINDOWS (POWERSHELL / CMD)

```
nslookup sakura.proxy.rlwy.net  
Non-authoritative answer:  
Address: 66.33.22.220
```

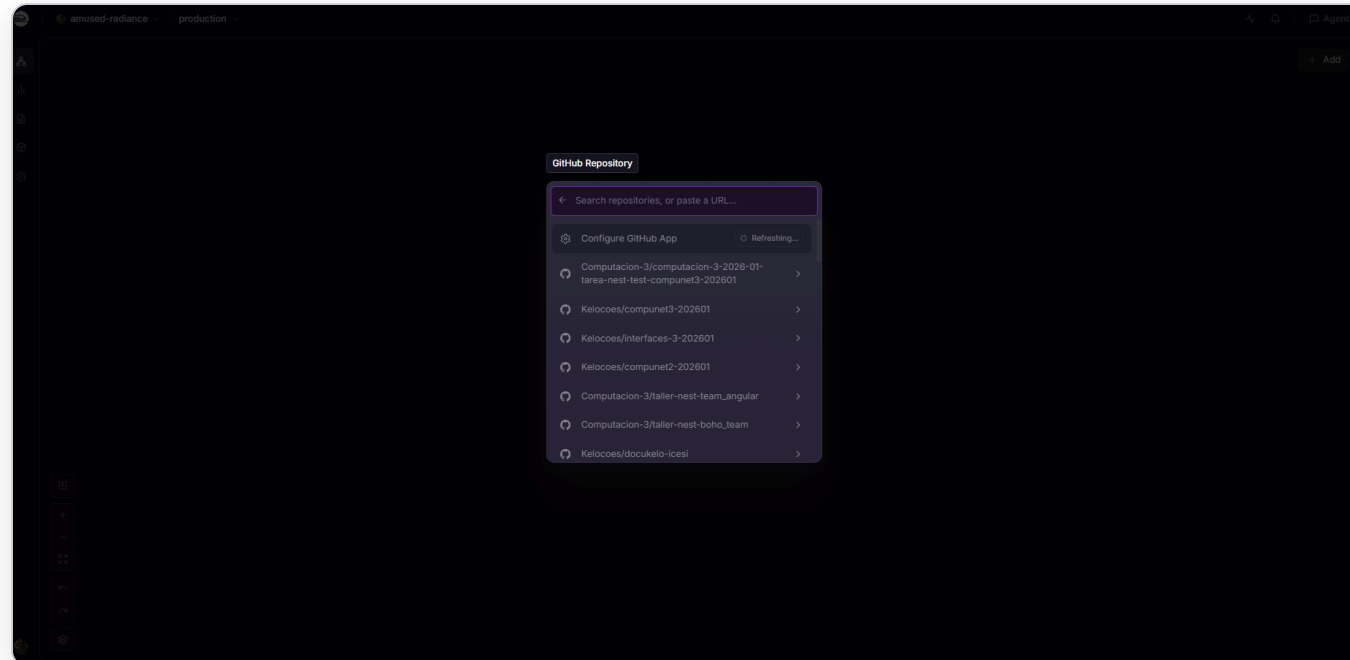
MACOS / LINUX TERMINAL

```
dig +short sakura.proxy.rlwy.net  
66.33.22.220
```

Paso 4: Conexión con Repositorio GitHub

Paso 4

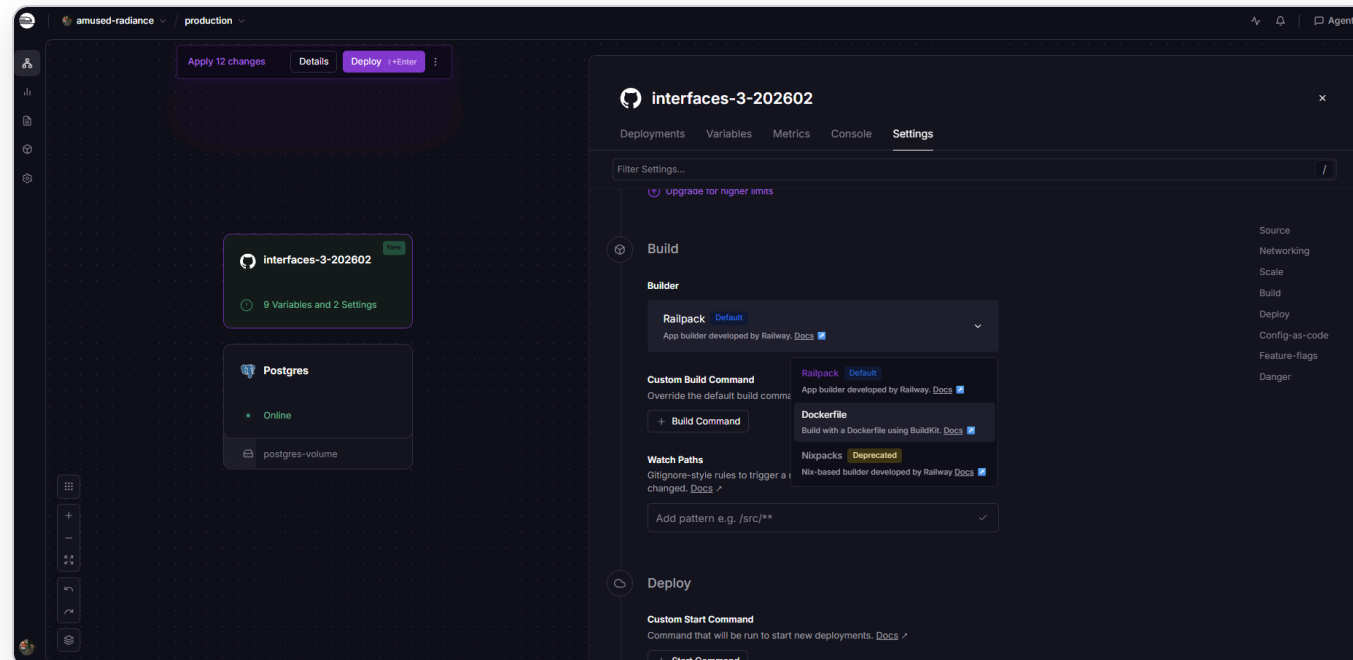
Pulsamos **+ New** → **GitHub Repo** y seleccionamos el repositorio del backend NestJS. Railway vinculará webhooks automáticos para compilar y desplegar (CI/CD) ante cada `git push`.



Paso 5: Configuración de Build y Dockerfile

Paso 5

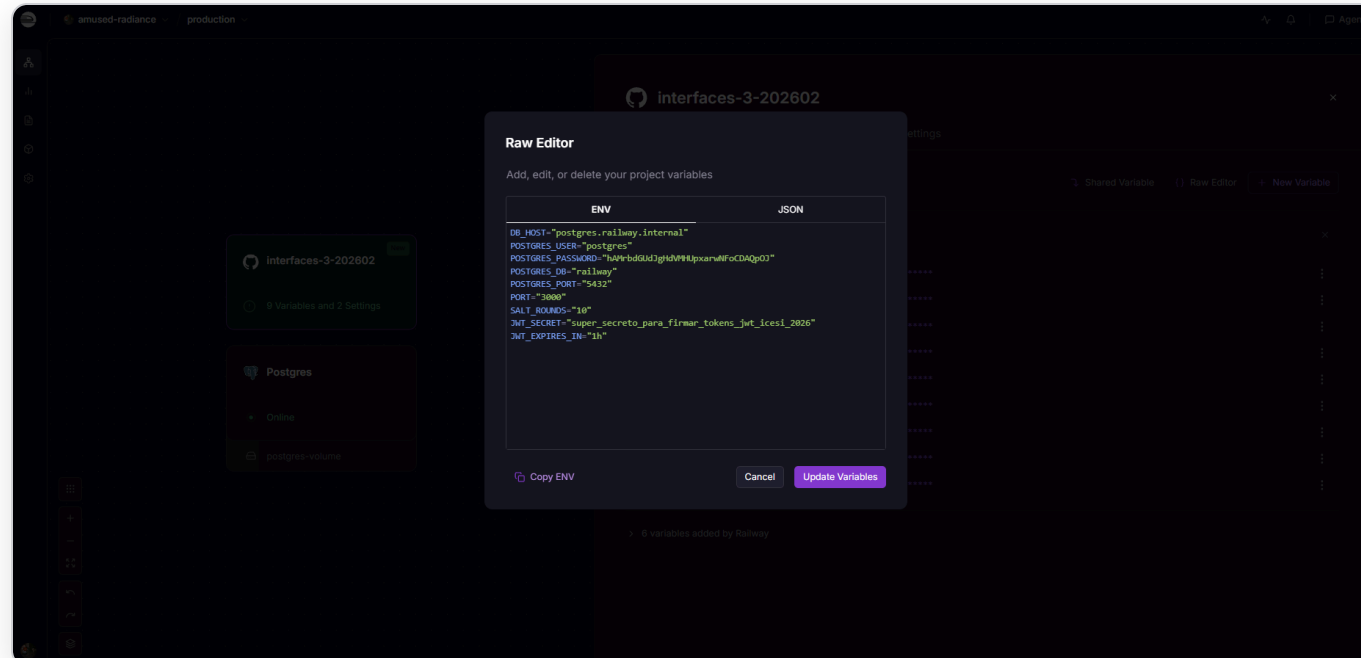
En la pestaña **Settings** → **Build** confirmamos que el Builder esté configurado como **Dockerfile**. Railway ejecutará las fases `deps`, `builder` y `runner` de nuestro Dockerfile multi-stage.



Paso 6: Inyección de Secretos con Raw Editor

Paso 6

En la sección **Variables** pulsamos **Raw Editor** para inyectar en bloque las variables operativas (`PORT` , `JWT_SECRET` , `SALT_ROUNDS`) y conectar las referencias privadas de la base de datos PostgreSQL.



Buenas Prácticas de Despliegue

IMÁGENES MÍNIMAS

Multi-Stage Build

Utilizar imágenes base Alpine, omitir herramientas de compilación en runner (`--omit=dev`) y mantener la imagen final por debajo de 150 MB.

SEGURIDAD DE SECRETOS

Zero Hardcoding

Jamás almacenar contraseñas, cadenas de conexión o secretos en el repositorio Git. Inyectarlos como variables protegidas en Railway.

RED PRIVADA AISLADA

Railway Private Mesh

Conectar backend y base de datos mediante el host privado interno. Sin tráfico expuesto a Internet ni cargos de transferencia de salida (egress).

INTEGRACIÓN CONTINUA

CI/CD Automático

Configurar despliegues continuos desde la rama `main`. Cada push genera una compilación y despliegue sin caída de servicio (Zero-Downtime).

¿Preguntas?

Desarrollo de Entornos Digitales Web — Universidad Icesi